



TERRACLOUD

CONTINUOUS CONFIGURATION AUTOMATION



TERRACLOUD

As a member of an infrastructure team, you've been tasked with building Cloud infrastructures.



Your main goals are:

- ✓ to deploy an application in an Azure resource group;
- ✓ to investigate and compare two models of cloud computing
- ✓ compare performance of the deployed infrastructures in terms of cost and scalability ;
- ✓ be able to reliably destroy and rebuild the infrastructures using IaC techniques to ensure reproducibility and efficient management of Cloud environments ;
- ✓ manage your different environment states and save them ;
- ✓ automate the deployment process to spin up and tear down the infrastructure on-demand.

You are expected to:

- ✓ deploy a highly available application on Azure ;
- ✓ configure and deploy an **I**nfrastructure **as a S**ervice infrastructure, using Azure virtual machines ;
- ✓ configure and deploy a **P**latform **as a S**ervice infrastructure, using the appropriate Azure services ;
- ✓ compare performance of the deployed infrastructures, in terms of cost and scalability ;
- ✓ ensure reproducibility and efficient management of Cloud environments (ie: you must be able to reliably destroy and rebuild the infrastructures using **IaC** techniques) ;
- ✓ automate the deployment process to spin up and tear down the infrastructure on-demand.

You can download the application provided with the assignment. You will also need to add a CI mechanism to create a Docker image, for example.



You must use [Terraform](#) for create ressource in your ressource groupe Azure.



HashiCorp

Terraform

Your manager would like to take the opportunity to test some solutions :

- ✓ Azure Web Apps ;
- ✓ Ansible ;
- ✓ Docker.



You'll have access to ressource groupe environment via the [Azure Portal](#). It allows you to deploy one virtual machines and one services app.

However, it's important to note that there are certain limits and restrictions in place to ensure efficient resource utilization and adherence to azure policies.



Note that some services can be only be deployed using the Portal or terraform.

Because the resources deployed uses a live subscription, you MUST manage them responsibly. You'll have to ensure cost optimization and compliance by leveraging Infrastructure as Code (IaC) and configuration tools.



We strongly advise shutting down all virtual machines at the end of each day to conserve resources and minimize costs.



You will have to manage different environment with different infrastructure, and manage your Terraform state file to create and destroy your resources.

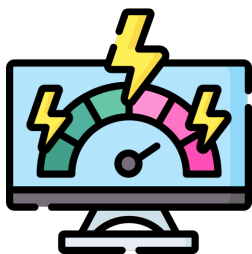
For the PaaS component, you will have to deploy a “Web App” to deploy the provided application.

For the **IaaS** deployment, choose your preferred operating system. You’ll have to deploy resources and a sample application using Terraform, Ansible and Docker.

For the **PaaS** component, you’ll have to deploy a “Web App” as a sample application.

Your manager ask you to:

- ✓ write a guide for both deployment ;
- ✓ provide a way to easily redeploy an environment ;
- ✓ provide “best practice” in resources management, like tags, naming convention, ...



You must perform stress tests and compare the app performances.



You can investigate, at least, the response status and time.

You also have to investigate the costs of each infrastructure model.



Compare the resource consumptions, but also the public prices.



Bonus

You must ensure that anybody can manage actual resources and deploy new one, by storing your state file and share code efficiently.

He also ask you to compare performance of the application, by mesuring response status and time, and stress it. You also have to compare cost by comparing resource consumption and public prices.

Then you can continue with :

- ✓ automate the deployment process (*using Github Actions for instance*) to spin up and tear down the infrastructure on-demand ;
- ✓ create some Azure VM Template to hasten deployments ;
- ✓ set up some monitoring tools to prevent and/or warn for anomalies ;
- ✓ use terragrunt to deploy multiple environments and keep IaC code DRY.

{EPITECH}

